**Bahasa Pemrograman Tingkat Rendah (MWCA010)**

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

# Register 8086 dan Register Flag

Pertemuan 3

---

Dosen Pengampu

**Cecep Suwanda, S.Si., M.Kom.**

Semester Ganjil

# Capaian dan Tujuan Pembelajaran

## Sub-CPMK 1.2

Mengidentifikasi fungsi register umum (`AX`, `BX`, `CX`, `DX`), register segmen (`CS`, `DS`, `SS`, `ES`), register *pointer*/*index* (`SP`, `BP`, `SI`, `DI`), dan register *flag* beserta pengaruhnya terhadap eksekusi program.

**CPMK yang didukung: CPMK01** ▪ **CPL yang didukung: CPL02**

## Setelah pertemuan ini, mahasiswa mampu:

1. menyebutkan register umum 8086 beserta akses byte rendah dan byte tingginya;
2. menjelaskan peran khas setiap register umum dan alasan di balik peran tersebut;
3. menjelaskan fungsi register segmen serta peran register *pointer*, *index*, dan `IP`;
4. mengidentifikasi sembilan *flag* pada register `FLAGS` beserta posisi bitnya;
5. memprediksi perubahan *flag* akibat operasi aritmatika dan menjelaskan akibatnya bagi alur program.

# Peta Materi

1. **Register Umum dan Register Segmen**
   `AX`–`DX` dan akses byte rendah/tinggi; peran khas tiap register; `CS`, `DS`, `SS`, `ES`
2. **Register *Pointer*, *Index*, dan *Instruction Pointer***
   `SP` dan `BP` pada *stack*; `SI` dan `DI` pada operasi string; `IP`; register yang rawan diubah
3. **Register Flag**
   peta bit `FLAGS`; sembilan *flag* status dan kendali; contoh perubahan *flag*; lompatan bersyarat

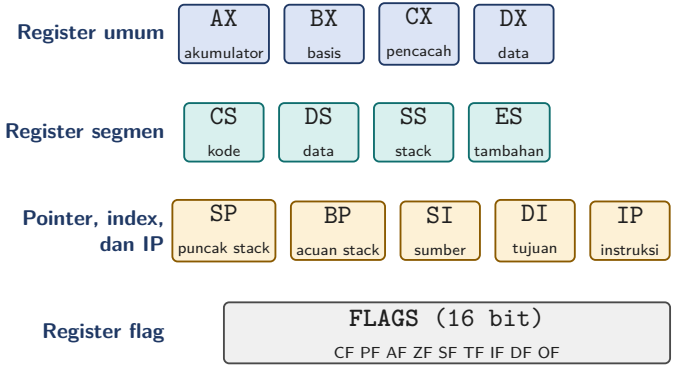
## Praktik pada pertemuan ini

Menjalankan tiga contoh program, memprediksi *flag*, lalu mengamati register melalui *Single Step*.

BAGIAN

# Register Umum dan Register Segmen

# Empat Belas Register, Empat Kelompok




Keempat belas register berukuran 16 bit dan dapat menampung nilai `0000h` sampai `FFFFh`. Register adalah memori tercepat di dalam prosesor: satu daur mesin, tanpa melewati bus.

Sumber: Bab V, Ringkasan kelompok register 8086

# Register Umum dan Akses Byte

| Register | Byte tinggi | Byte rendah | Peran khas |
|---|---|---|---|
| AX | AH | AL | Akumulator: hasil hitungan, operand wajib MUL/DIV, layanan DOS |
| BX | BH | BL | Basis: penunjuk alamat memori pada mode terindeks |
| CX | CH | CL | Pencacah: pengulangan LOOP, operasi string, jumlah pergeseran SHL/SHR |
| DX | DH | DL | Data: pasangan AX pada operasi 32 bit, alamat port I/O dan layanan DOS |

Setiap register 16 bit terbagi menjadi *byte tinggi* dan *byte rendah* yang dapat diakses sendiri-sendiri.

Sumber: Bab V, Register Umum dan Akses Byte

# Akses Byte: `MOV AL, 5` dan `MOV AX, 5`

Dua instruksi ini terlihat serupa, tetapi akibatnya berbeda pada `AH`.

```
MOV AL, 5   ; hanya byte rendah yang berubah
MOV AX, 5   ; kedua bagian ditulis sekaligus
```

**Isi `AX`**

```
sebelum       : FF00h
MOV AL, 5     : FF05h
MOV AX, 5     : 0005h
```

## Yang sering terlewat

`MOV AL, 5` **tidak** mengosongkan `AH`. Bila `AH` semula `FFh`, maka `AX` menjadi `FF05h`, bukan `0005h`.

# Mengapa Peran Setiap Register Berbeda

Keempat register umum sama-sama dapat menampung nilai apa pun, tetapi peran khasnya berbeda karena cara prosesor 8086 menyandikan instruksi.

| Register | Peran | Mengapa demikian |
|---|---|---|
| `AX` | Akumulator | `MUL`, `DIV`, `IN`, `OUT`, dan sebagian besar layanan DOS mengandaikan operand di `AX`/`AL`; bentuk mesinnya juga lebih pendek |
| `BX` | Basis | Hanya `BX` yang boleh menjadi register basis di dalam tanda kurung: `MOV AL, [BX]` |
| `CX` | Pencacah | `LOOP`, `REP`, dan operasi string memakainya sebagai penghitung otomatis; `SHL`/`SHR` memakai `CL` |
| `DX` | Data | Pasangan `AX` pada operasi 32 bit — `DX` menyimpan bagian tinggi — serta alamat string dan nomor port |

Register umum boleh dipakai di luar peran khasnya; yang tidak boleh adalah melanggar aturan khusus, misalnya `MOV AX, [CX]`.

# Register Segmen

| Register | Nama lengkap | Peran |
|---|---|---|
| `CS` | *Code Segment* | Menunjuk segmen tempat instruksi berada; selalu berpasangan dengan `IP` |
| `DS` | *Data Segment* | Menunjuk segmen tempat variabel berada; dipakai pada hampir semua akses data |
| `SS` | *Stack Segment* | Menunjuk segmen tempat *stack* berada; berpasangan dengan `SP` dan `BP` |
| `ES` | *Extra Segment* | Segmen tambahan; menjadi segmen tujuan operasi string bersama `DI` |

## Yang disimpan register segmen

Bukan data yang dihitung, melainkan *nomor segmen* tempat data atau instruksi berada.

Sumber: Bab V, Register Segmen

# Pasangan Segmen–Offset dan Inisialisasi DS

Setiap akses memori bermain dengan pasangan register segmen–offset:

- CS:IP — alamat instruksi berikutnya.
- DS:SI atau DS:BX — alamat data umum.
- SS:SP atau SS:BP — alamat di dalam *stack*.
- ES:DI — alamat tujuan operasi string.

## Dua kesalahan umum

1. `MOV DS, 1000h` **tidak sah**. Nilai hanya boleh dari register umum: `MOV AX, 1000h` lalu `MOV DS, AX`.
2. Mengubah DS tanpa menyimpan nilai lamanya, sehingga variabel pada segmen data lama tidak terjangkau.

Karena itu program .EXE wajib menginisialisasi DS di awal program.

# Rangkuman Bagian 1

- Prosesor 8086 memiliki empat belas register 16 bit: empat register umum, empat register segmen, empat register *pointer*/*index*, ditambah `IP` dan `FLAGS`.
- `AX`, `BX`, `CX`, dan `DX` dapat diakses sebagai dua byte terpisah. Menulis ke byte rendah **tidak** mengubah byte tinggi.
- Peran khas muncul dari cara instruksi disandikan: `AX` akumulator, `BX` basis pengalamatan, `CX` pencacah, `DX` pasangan data dan penunjuk port.
- Register segmen menyimpan nomor segmen. Pasangan bakunya: `CS`:`IP`, `DS`:`SI`, `SS`:`SP`, `ES`:`DI`.

BAGIAN

# Register Pointer, Index, dan Instruction Pointer

# Stack Pointer dan Base Pointer

`SP` (*Stack Pointer*) Selalu menunjuk puncak *stack* saat ini. Setiap `PUSH` mengurangi `SP` sebesar 2 dan setiap `POP` menambahnya sebesar 2, karena satu kata berukuran 2 byte. Nilainya berpasangan dengan `SS`.

`BP` (*Base Pointer*) Menjadi titik acuan tetap di dalam *stack*. Berbeda dengan `SP` yang terus bergerak, `BP` disetel sekali di awal prosedur lalu dipakai untuk menjangkau parameter dan variabel lokal.

**Akibat mengubah `SP` sembarangan**

Puncak *stack* ikut berpindah, sehingga instruksi `RET` dapat melompat ke alamat yang salah.

Sumber: Bab V, Stack Pointer dan Base Pointer

# Source Index dan Destination Index

Dua register ini dipakai untuk mengakses deretan data, misalnya *string* atau *array*.

## SI — *Source Index*

Menunjuk alamat *sumber* pada operasi string. Pasangannya DS, sehingga alamatnya ditulis DS:SI.

## DI — *Destination Index*

Menunjuk alamat *tujuan* pada operasi string. Pasangannya ES, sehingga alamatnya ditulis ES:DI.

## Memindahkan data antar segmen

MOVSB, MOVSW, dan REP MOVSB memindahkan data dari DS:SI ke ES:DI sambil menyesuaikan kedua register secara otomatis — karena itu keduanya harus diisi lebih dahulu.

# Instruction Pointer dan Register yang Rawan

## IP — *Instruction Pointer*

- Menyimpan *offset* instruksi berikutnya yang akan diambil; alamat lengkapnya ditentukan pasangan CS:IP.
- Nilainya bertambah sendiri setiap kali satu instruksi selesai dijalankan.
- Programmer hampir tidak pernah menulis IP langsung; perubahannya terjadi melalui JMP, CALL, RET, dan INT.

## Register yang tidak boleh diubah sembarangan

CS dan IP menentukan instruksi berikutnya; SS dan SP menentukan letak *stack*. Bila salah satunya diubah tanpa alasan benar, prosesor dapat mengambil instruksi dari alamat salah, menjalankan data sebagai instruksi, atau membuat program berhenti. DS boleh diubah, tetapi harus dikembalikan sebelum variabel lama diakses.

# Rangkuman Bagian 2

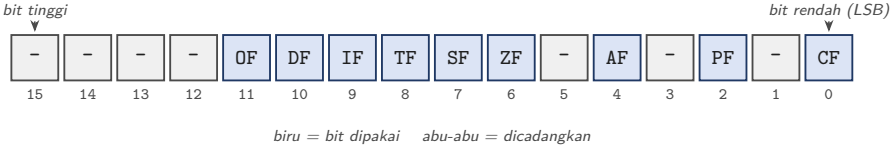
- Register *pointer* dan *index* menyimpan *offset*, yaitu jarak sebuah lokasi dari awal segmen.
- `SP` menunjuk puncak *stack* dan berubah otomatis sebesar 2 pada `PUSH` dan `POP`; `BP` menjadi acuan tetap untuk parameter dan variabel lokal.
- `SI` menunjuk sumber (`DS:SI`) dan `DI` menunjuk tujuan (`ES:DI`) pada operasi string.
- `IP` menyimpan *offset* instruksi berikutnya dan selalu berpasangan dengan `CS`.
- `CS`, `IP`, `SS`, dan `SP` hanya diubah melalui instruksi yang memang dirancang untuk itu.

BAGIAN

# Register Flag

# Peta Bit Register `FLAGS`

Register `FLAGS` 16 bit — disebut juga *Program Status Word* (PSW) — tidak menyimpan angka hasil hitungan, melainkan keterangan keadaan hasil operasi terakhir.




Dari 16 bit yang tersedia, hanya sembilan bit yang dipakai; sisanya dicadangkan dan nilainya tidak ditentukan.

Sumber: Bab V, Peta Bit Register Flag

# Sembilan Flag: Status dan Kendali

| Bit | Flag | Kelompok | Arti singkat |
|---|---|---|---|
| 0 | `CF` | Status | *Carry*: simpanan keluar dari bit paling tinggi |
| 2 | `PF` | Status | *Parity*: paritas byte rendah hasil operasi |
| 4 | `AF` | Status | *Auxiliary Carry*: simpanan dari nibble rendah (bit ke-3) |
| 6 | `ZF` | Status | *Zero*: hasil operasi bernilai nol |
| 7 | `SF` | Status | *Sign*: bit tanda hasil menyala |
| 8 | `TF` | Kendali | *Trap*: satu instruksi per langkah untuk *debugging* |
| 9 | `IF` | Kendali | *Interrupt*: izin menerima *interrupt* tertutup |
| 10 | `DF` | Kendali | *Direction*: arah gerak `SI`/`DI` pada operasi string |
| 11 | `OF` | Status | *Overflow*: hasil bertanda di luar jangkauan |
| 1, 3, 5, 12–15 | — | Cadangan | Tidak dipakai |

# Enam Flag Status: Kapan Bernilai 1

| Flag | Bernilai 1 bila ... | Contoh instruksi |
|---|---|---|
| CF | Operasi *tak bertanda* menghasilkan simpanan keluar dari bit tertinggi | ADD, SUB, CMP, ADC, SBB, NEG |
| PF | Byte rendah hasil memuat jumlah bit 1 yang *genap* | Semua operasi aritmatika dan logika |
| AF | Simpanan keluar dari bit ke-3 (nibble rendah) | ADD, ADC, SUB, SBB, CMP, INC, DEC |
| ZF | Hasil operasi tepat sama dengan nol | ADD, SUB, CMP, AND, OR, XOR, TEST |
| SF | Bit paling tinggi hasil operasi bernilai 1 | ADD, SUB, CMP, AND, OR, XOR, NEG |
| OF | Hasil *bertanda* di luar jangkauan: $-128 \ldots 127$ untuk byte | ADD, SUB, CMP, IMUL, IDIV, NEG |

TF, IF, dan DF adalah *flag* kendali: nilainya ditentukan programmer melalui instruksi seperti STI, CLI, STD, dan CLD.

Sumber: Bab V, Arti dan Pengaruh Setiap Flag

# Tiga Kasus `ADD AL, BL`

| AL | BL | AL baru | CF | OF | SF | **Alasan singkat** |
|---|---|---|---|---|---|---|
| 7Fh | 01h | 80h | – | 1 | 1 | Tanpa simpanan keluar, tetapi penjumlahan bertanda melampaui batas 127 |
| 80h | 80h | 00h | 1 | 1 | – | Simpanan keluar terjadi; dua bilangan negatif menghasilkan nol |
| FFh | 01h | 00h | 1 | – | – | Simpanan keluar terjadi, tetapi hasil bertanda tepat nol |

- `7Fh` + `01h`: dua operand positif menghasilkan hasil yang ditafsirkan negatif → `OF` = 1.
- `80h` + `80h`: 128 + 128 = 256 → `CF` = 1 dan `ZF` = 1.
- `FFh` + `01h`: `FFh` adalah −1 dan `01h` adalah +1, sehingga hasil bertandanya nol → `OF` = 0.

Sumber: Bab V, Contoh Perubahan Flag pada ADD AL, BL

# CF dan OF Menjawab Pertanyaan Berbeda

## Jangan sampai tertukar

CF berlaku untuk bilangan **tak bertanda**, sedangkan OF berlaku untuk bilangan **bertanda**. Memakai JC untuk memeriksa hasil bilangan bertanda adalah salah; yang tepat JO atau JS.

## MOV dan kawan-kawan tidak menyentuh *flag*

MOV, LEA, PUSH, dan POP **tidak** mengubah *flag*. Pengecualiannya POPF dan SAHF yang memang dirancang untuk menulis *flag*.

## INC dan DEC juga bukan pengubah penuh

Keduanya mengubah ZF, SF, OF, dan AF, tetapi membiarkan CF apa adanya. Untuk menambah 1 sekaligus memperbarui CF, pakai `ADD AL, 1`.

# Flag Dibaca oleh Lompatan Bersyarat

*Flag* baru bermakna ketika dibaca lompatan bersyarat. `CMP` membandingkan dua nilai tanpa menyimpan hasil, lalu meninggalkan jejaknya pada *flag*.

| Flag | Lompatan | Makna |
|---|---|---|
| `ZF` | `JE, JZ` | Lompat bila hasil nol (kedua nilai sama) |
| `ZF` | `JNE, JNZ` | Lompat bila hasil tidak nol (kedua nilai berbeda) |
| `CF` | `JC, JB` | Lompat bila ada simpanan keluar (nilai pertama lebih kecil secara tak bertanda) |
| `CF` | `JNC, JAE` | Lompat bila tidak ada simpanan keluar |
| `SF` | `JS, JNS` | Lompat bila bit tanda menyala atau tidak |
| `OF` | `JO, JNO` | Lompat bila terjadi *overflow* bertanda atau tidak |
| `PF` | `JP, JNP` | Lompat berdasarkan paritas byte rendah hasil |

Sumber: Bab V, Pengaruh Flag terhadap Keputusan Program

# Rangkuman Bagian 3

- FLAGS 16 bit memuat sembilan *flag* aktif: enam *flag* status (CF, PF, AF, ZF, SF, OF) dan tiga *flag* kendali (TF, IF, DF).
- CF melaporkan simpanan keluar bilangan tak bertanda; OF melaporkan *overflow* bilangan bertanda. Keduanya tidak boleh dipertukarkan.
- ZF menandai hasil nol, SF bit tanda yang menyala, PF paritas byte rendah, dan AF simpanan keluar dari nibble rendah.
- MOV, LEA, PUSH, dan POP tidak mengubah *flag*; INC dan DEC mengubah semuanya kecuali CF.
- *Flag* dipakai lompatan bersyarat: JE/JNE, JC/JNC, JS/JNS, JO/JNO, dan JP/JNP.

# Contoh 1: Memprediksi Flag Sebelum Menjalankan Program

Dua penjumlahan yang sengaja dipilih untuk memicu kondisi *flag* berbeda.

```
.DATA
    hasil1 DW 0
    hasil2 DW 0
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    MOV AX, 007Fh       ; kasus 1: +127
    MOV BX, 0001h
    ADD AX, BX          ; AX=0080h: SF=1, OF=1
    MOV hasil1, AX      ; MOV tak mengubah flag
    MOV AX, 8000h       ; kasus 2: -32768
    MOV BX, 8000h
    ADD AX, BX          ; AX=0000h: CF=1, ZF=1
    MOV hasil2, AX
```

## Keluaran

```
Pengamatan flag
selesai.
```

CF, OF, SF, dan ZF diprediksi dahulu, lalu diperiksa dengan *Single Step*.

`prediksi-flag.asm`

# Contoh 2: Memilih Pesan Berdasarkan Flag

F0h ditambah 10h menghasilkan 100h: AL menyimpan 00h dengan CF = 1 dan ZF = 1.

```
    MOV AL, 0F0h        ; AL = 1111 0000b
    MOV BL, 10h         ; BL = 0001 0000b
    ADD AL, BL          ; AL=00h, CF=1, ZF=1

    JNC tanpa_carry     ; lompat bila CF = 0
    MOV AH, 09h         ; cetak pesan carry
    LEA DX, pesan_carry
    INT 21h
tanpa_carry:
    JNZ bukan_nol       ; lompat bila ZF = 0
    MOV AH, 09h         ; cetak pesan hasil nol
    LEA DX, pesan_nol
    INT 21h
bukan_nol:
    JNS tanpa_negatif   ; lompat bila SF = 0
```

## Keluaran

```
Terjadi carry,
CF = 1.
Hasil penjumlahan
nol, ZF = 1.
```

Layanan INT 21h tidak merusak *flag*: INT menyimpan FLAGS ke *stack* dan IRET mengembalikannya.

pesan-flag.asm

# Contoh 3: Akses Byte Rendah dan Byte Tinggi

Satu register 16 bit dapat diperlakukan sebagai dua register 8 bit yang mandiri.

```
.MODEL SMALL
.STACK 100h

.DATA
    dummy DB 0          ; agar segmen data ada

.CODE
START:
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, 1234h       ; AH = 12h, AL = 34h
    MOV BL, AH          ; BL = 12h
    MOV BH, AL          ; BH = 34h
    ADD AL, BL          ; AL = 46h, AH tetap
    MOV CL, AL          ; CL = 46h
```

**Keadaan akhir**

```
AX = 1246h
BX = 3412h
CL = 46h
```

`ADD AL, BL` hanya mengubah `AL`. Nilai `AH` tetap `12h` karena operasi 8 bit pada `AL` tidak menyentuh byte tinggi.

# Praktikum: Pengamatan Register dan Flag

## Judul praktikum

Pengamatan register dan *flag* dengan *Single Step* pada Emu8086 ▪ **Sub-CPMK:** 1.2 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu

### Alat dan bahan

- Komputer dengan Emu8086 terpasang
- Berkas `prediksi-flag.asm` dan `pesan-flag.asm`
- Lembar kerja pengamatan register dan *flag*

### Langkah kerja

1. Buka jendela `Registers` dan `Flags`, lalu jalankan dengan *Single Step*
2. Catat `AX`, `BX`, `SP`, dan `IP`; amati *flag* sebelum dan sesudah `ADD AX, BX`
3. Ubah `AX` menjadi `FFFFh` dan `AL` menjadi `00h` secara manual
4. Ubah `CF` dan `ZF` pada `pesan-flag.asm`, amati pesan yang tercetak

**Dikumpulkan:** lembar pengamatan register, lembar prediksi *flag*, tangkapan layar `Registers` dan `Flags`, serta catatan percobaan mengubah `CF` dan `ZF`.

# Latihan

1. Jelaskan apa yang dimaksud dengan register, lalu sebutkan empat kelompok register 8086 beserta satu contoh register pada setiap kelompok.

2. `AX` berisi `ABCDh`. Tentukan nilai `AH` dan `AL` setelah `MOV AL, 55h` dijalankan. Mengapa `AH` tidak berubah?

3. Seorang programmer ingin menjumlahkan dua nilai, menyimpan hasilnya, lalu mengulangi proses sepuluh kali. Register mana yang tepat untuk menampung hasil dan mana yang tepat menjadi pencacah? Jelaskan alasannya.

4. Untuk `ADD AL, BL`, tentukan `CF`, `OF`, `SF`, dan `ZF` bila `AL` dan `BL` bernilai (*a*) `7Fh` dan `01h`, (*b*) `05h` dan `05h`. Sertakan penjelasan singkat.

5. Sebuah program berisi `MOV SP, 0000h` sebelum memanggil prosedur. Jelaskan mengapa berbahaya dan apa yang terjadi saat `RET` dijalankan.

# Rangkuman

- Register adalah tempat penyimpanan tercepat di dalam prosesor. Prosesor 8086 memiliki empat belas register 16 bit yang dapat dipakai programmer.
- `AX`, `BX`, `CX`, dan `DX` dapat diakses sebagai dua byte terpisah. Menulis ke byte rendah tidak mengubah byte tinggi.
- Peran khas muncul dari cara instruksi disandikan: `AX` akumulator, `BX` basis pengalamatan, `CX` pencacah, `DX` pasangan data dan penunjuk port.
- Register segmen menyimpan nomor segmen, bukan data; pasangannya `IP` untuk kode, `SP`/`BP` untuk *stack*, serta `SI` dan `DI` untuk data.
- `FLAGS` memuat sembilan *flag* aktif: enam status dan tiga kendali. `CF` untuk bilangan tak bertanda, `OF` untuk bilangan bertanda.
- `CS`, `IP`, `SS`, dan `SP` mengendalikan alur program dan letak *stack*, sehingga tidak boleh diubah sembarangan.

# Cek Pemahaman

1. Setelah `MOV AL, 20h` dijalankan pada `AX` yang semula berisi `1234h`, isi `AX` menjadi ...
   - 1.1 `1220h`
   - 1.2 `2034h`
   - 1.3 `0020h`
   - 1.4 `2020h`

2. Register yang selalu berpasangan dengan `CS` untuk menentukan alamat instruksi berikutnya adalah ...
   - 2.1 `SP`
   - 2.2 `IP`
   - 2.3 `DS`
   - 2.4 `DI`

3. `ADD AL, BL` dengan `AL` = `7Fh` dan `BL` = `01h` menghasilkan ...
   - 3.1 `CF` = 1 dan `OF` = 0
   - 3.2 `CF` = 0 dan `OF` = 1
   - 3.3 `CF` = 1 dan `OF` = 1
   - 3.4 `CF` = 0 dan `OF` = 0

# Penutup

## Sebelum pertemuan berikutnya

- Selesaikan praktikum: lembar pengamatan register dan lembar prediksi *flag*.
- Tuliskan potongan program yang memuat `F0h` ke `AL` dan `10h` ke `BL`, lalu cetak "NOL" bila `ZF` bernilai 1.
- Baca **Bab VI** — Segmentasi Memori dan Alamat Fisik.

**Terima kasih**

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil